

PRACTICAL WEEK_06

1. Create a client- server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneous

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main()
{
    char msg[100];
    char reply[100];

    int fd1 = open("fifo1", O_WRONLY);

    printf("Enter Message: ");
    fgets(msg, sizeof(msg), stdin);

    write(fd1, msg, strlen(msg)+1);
    close(fd1);

    int fd2 = open("fifo2", O_RDONLY);

    read(fd2, reply, sizeof(reply));

    printf("Server Response: %s\n", reply);

    close(fd2);

    return 0;
}
```

ly.

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    char msg[100];

    mkfifo("fifo1", 0666);
    mkfifo("fifo2", 0666);

    while(1)
    {
        int fd1 = open("fifo1", O_RDONLY);
        read(fd1, msg, sizeof(msg));

        printf("Client: %s\n", msg);

        close(fd1);

        int fd2 = open("fifo2", O_WRONLY);

        char reply[] = "Message received by server";
        write(fd2, reply, strlen(reply)+1);

        close(fd2);
    }

    return 0;
}

```

CODE: 1

RESULT:

```

root@SRIVALLI: ~# ./server
Client: hi srivalli

root@SRIVALLI: ~# ./client
Enter Message: hi srivalli
Server Response: Message received by server
root@SRIVALLI: ~#

```

REPORT:

In this experiment, a client-server application was developed using Named Pipes (FIFOs) in Ubuntu Linux. The client sends messages to the server through one FIFO, and the server processes the messages and sends responses back through another FIFO. This demonstrates Inter-Process Communication (IPC) between independent processes. The experiment also shows that when multiple clients communicate with the server simultaneously, the FIFO follows the First In First Out principle, processing

messages in the order they arrive. Thus, named pipes provide a simple and effective mechanism for client-server communication in Linux.

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

CODE:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void signalHandler(int sig)
{
    if(sig == SIGINT)
        printf("\nSIGINT received (Ctrl+C)\n");

    else if(sig == SIGTERM)
        printf("\nSIGTERM received\n");

    else if(sig == SIGUSR1)
        printf("\nSIGUSR1 received\n");
}

int main()
{
    signal(SIGINT, signalHandler);
    signal(SIGTERM, signalHandler);
    signal(SIGUSR1, signalHandler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while(1)
    {
        sleep(1);
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano signal_handler.c
root@SRIVALLI:~# gcc signal_handler.c -o signal_handler
root@SRIVALLI:~# ./signal_handler
Process ID: 528
Waiting for signals...
^C
SIGINT received (Ctrl+C)

root@SRIVALLI:~# kill -SIGUSR1 PID
-bash: kill: PID: arguments must be process or job IDs
root@SRIVALLI:~#
```

REPORT:

In this experiment, a POSIX signal handling program was developed to capture and process the signals SIGINT, SIGTERM, and SIGUSR1. Signal handlers were defined and registered using the signal() system call. When a signal is received, the corresponding handler executes and

displays a message without interrupting the normal flow of the program. SIGINT is generated using Ctrl+C, while SIGTERM and SIGUSR1 are sent using the kill command. This experiment demonstrates asynchronous event handling, where signals can occur at any time during program execution. It also helps in understanding the role of signal handlers in managing events and controlling process behavior in Linux systems.